

1. Autenticación

1.1. Qué es autenticar

Definición: Autenticación

Asociar una **identidad** a un **principal**. La identidad corresponde a una entidad externa (la entidad del entorno), y el principal es la representación interna del sistema (p. ej. “Usuario1”, “Sistema1”). El componente que realiza la asociación es el **autenticador**.

La entidad externa es generalmente física y debe representarse de alguna forma dentro del sistema. Esa representación interna se llama *principal* (típicamente el ID de usuario o algún identificador). Cada vez que un sujeto quiere autenticarse hay que comprobar que sea quien dice ser y no alguien que lo **impersona**.

Pasos de la confirmación de identidad.

1. Obtener información de identidad.
2. Analizarla (ver si es fidedigna).
3. Determinar si corresponde a una entidad del sistema.

Consecuencias: hay que almacenar información de cada entidad y contar con mecanismos para procesar esa información.

1.1.1. Componentes de un sistema de autenticación

Análogo a un criptosistema, es una tupla de **5 componentes**:

Comp.	Nombre	Descripción
$A = \{a\}$	Información de autenticación	Designación de la entidad e información provista. La provee la entidad externa; el sistema <i>no</i> la controla.
$C = \{c\}$	Información complementaria	Especificación de un principal + información almacenada por el sistema.
$F = \{f : A \rightarrow C\}$	Funciones de complementación	Derivan la información complementaria a partir de A .
$L = \{l : A \times C \rightarrow \{0, 1\}\}$	Funciones de autenticación	Determinan si un par (A, C) es una asociación válida.
$S = \{s\}$	Funciones de selección	Permiten crear y actualizar A y C (administrativas).

! Importante — remarcado en clase

Distinción F vs L (el profe la aclaró por pedido de un alumno):

- F *deriva* C a partir de la información de autenticación A (ej: $C = h(A)$, el hash de la clave).
- L *corroborar*: dada una *nueva* información de autenticación, dice si existe una C correspondiente almacenada en el sistema.

S son administrativas (crear/actualizar/borrar entidades), p. ej. `adduser()`, `removeuser()`, `passwd()`.

Es deseable que **de C no se pueda volver a A** , porque si tengo C podría saber a quién pertenece la información (o incluso ingresar al sistema).

► En el parcial

Función de complementación (2016). Dada información de autenticación e información complementaria, retorna V/F. *Cuidado:* eso describe a la función de **autenticación** $L (A \times C \rightarrow \{0,1\})$. La función de **complementación** F es $A \rightarrow C$ (deriva C). Tener clara la diferencia.

► En el parcial

Módulos de Autenticación vs Control de Acceso (2013). Diagrama:

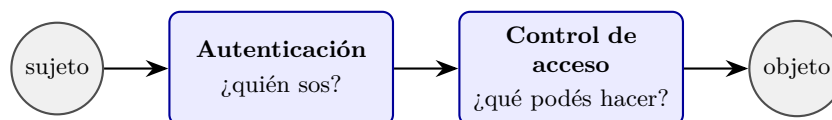
sujeto $\rightarrow$ módulo autenticación $\rightarrow$ módulo control de acceso $\rightarrow$ objeto

Piden dar una vulnerabilidad de *cada* módulo. Ideas del material:

- **Autenticación:** robo/adivinación de la clave (fuerza bruta, diccionario), impersonación, robo del segundo factor, robo de la base de información complementaria.
- **Control de acceso:** una vez impersonado un usuario, el sistema rara vez detecta que “ese no sos vos”; fallas en las reglas que definen si un sujeto accede a un objeto y de qué forma (MAC/DAC).

“Una vez que te personaste como otro usuario, ya está; el sistema, si no tiene métodos fuertes de detectar que ese no sos vos, por lo general no lo tiene, y podés hacer lo que quieras dentro del sistema.”

— dicho en clase



Dos módulos distintos: primero se verifica la identidad (autenticación), luego qué operaciones se permiten sobre el objeto (control de acceso). Cada módulo tiene sus propias vulnerabilidades.

1.2. Factores de autenticación

La información de autenticación posee distintos **grados de confianza**. Los *factores* indican la naturaleza de la información.

Factor	Descripción y ejemplos
Algo que conozco	Secreto compartido. Clave, frase (passphrase), pregunta secreta. Requiere almacenar de forma segura el secreto; depende de su confidencialidad. Es el mecanismo más empleado y base de otros.
Algo que tengo	Elemento físico. Smart card / USB token, generador de claves (RSA-ID), celular, tarjeta de coordenadas (token grid). Requiere cuidar el elemento; puede copiarse/clonarse; la info en tránsito puede ser suplantada.
Algo que soy	Características intrínsecas (biometría). Huella, retina, voz, cara. No fácilmente modificables. Métodos no 100 % eficaces; asumen que el dispositivo de lectura no puede manipularse.
Contexto	Red de origen, país/ciudad/geolocalización, host/terminal/device, día y hora, velocity. Suele usarse como complemento, no como único factor. Reglas positivas (operador desde la red privada del datacenter) o negativas (acceso desde otro país $\Rightarrow$ menos chances de ser quien dice).



Los tres factores clásicos de autenticación. A más factores independientes combinados, mayor grado de confianza.

! Importante — remarcado en clase

La información de autenticación la controla la entidad externa; el sistema no puede “ingerirla”/forzarla, pero sí puede **exigir ciertos factores** que gradúan la confianza. No es lo mismo que alguien diga una contraseña (que pudo haber escuchado) a que muestre su cara: distintos grados de confianza.

1.2.1. Biometría

No es 100 % eficaz por dos motivos: (1) no hay sistema de lectura siempre eficaz, y (2) aunque lo hubiera, la representación cambia (cambios físicos, etc.). Las primeras implementaciones de huella en celulares rechazaban al usuario; muchos sistemas **subieron el umbral de error** para no bloquear, aceptando lecturas imperfectas. Requiere un **mecanismo adicional** para evitar engaños (foto al lector de cara, manipular el dispositivo de lectura): se ponen cámaras u otros mecanismos para que no se manipule el lector.

! Importante — remarcado en clase

Puede pasar que la **misma función complementaria** valide *dos* informaciones de autenticación distintas: el caso real es el celular de una persona que se desbloquea con la cara de un hermano/primo parecido (falso positivo biométrico).

► En el parcial

Robo de identidad biométrico (V/F clásico, 2013). Verdadero: sí puede haber robo de identidad biométrico. Los datos biométricos pueden ser capturados/suplantados, el dispositivo de lectura manipulado, y existen falsos positivos (rasgos parecidos). La biometría no es infalible.

1.3. Claves (passwords)

A diferencia de las claves criptográficas, las passwords **las controla el usuario** y por lo general *no* son aleatorias. El sistema sí puede tomar decisiones sobre qué permite como clave.

Tipos.

- **Secuencias de caracteres:** con restricciones (largo, dígitos, mayúsculas, símbolos); generadas aleatoriamente, por el usuario o de manera asistida.
- **Secuencias de palabras (pass-phrases):** más largas. *Cuidado:* a veces son **más adivinables** que una contraseña normal (ej. “perro árbol 123”).
- **Algorítmicas:** challenge-response (pregunta-respuesta) y OTP (claves de uso único).

1.3.1. Almacenamiento de claves

Método	Comentario
Texto en claro	NO recomendado. Si roban la BD se llevan todo; accesible por fuera del sistema; los administradores/desarrolladores con acceso a la base podrían ver los secretos.
Archivo cifrado	Requiere una clave para acceder. La clave puede estar en config, en el ejecutable, ingresarse al iniciar, o en un dispositivo criptográfico. Solo recomendado si <i>se necesita</i> la clave original (p. ej. reenviarla a un sistema legacy).
Hash / func. de derivación	Forma correcta: guardar una <i>prueba</i> de que se conoce el secreto, sin guardar el secreto. Más abajo: salt + PBKDF2.

! Importante — remarcado en clase

Idea central del almacenamiento moderno: “*Se empieza a guardar como una prueba de que yo conozco el secreto, pero no guardo el secreto.*”

— dicho en clase La clave del usuario **nunca se almacena**; se valida sin guardarla.

Ejemplo: Unix legacy. Se almacena por cada usuario el resultado de **una de 4096** funciones de hash, elegida aleatoriamente.

- $A = \{\text{secuencias de hasta 8 caracteres}\}$
- $C = \{xx \underbrace{HHHHHHHHHHH}_{11}\}$, donde xx = función de hash (0–4095, 2 caracteres) y $HH...H$ = resultado (11 caracteres).
- $F = \{DES_{xx}(\cdot)\}$ (variante de DES, *no* reversible).
- $L = \{\text{login, su, sudo}\}; S = \{\text{passwd, adduser}\}.$

Marcó un precedente: hasta entonces no había consenso de cómo guardar contraseñas (se cifraban). Unix logró generar información complementaria de la que *no se puede volver* a la original.

1.3.2. Dispositivo criptográfico (HSM) y token físico

Las claves pueden guardarse en un **Hardware Security Module (HSM)** / cripto-chip especializado. Se usa cuando se requiere acceder a la clave original (contextos PCI, tarjetas de crédito, sistemas legacy de bancos que exigen la contraseña original).

► En el parcial

Token / llave física como segundo factor. Sirve porque la **clave privada no se puede extraer del hardware** (HSM/cripto-chip): “la clave nunca sale de ahí”. En los celulares hay un chip dedicado a biometría, por fuera del kernel del SO; el kernel solo recibe **OK/fail**, no accede a los datos biométricos.

1.4. Ataques a un sistema de autenticación

Objetivo del atacante: ser identificado como una entidad (impersonarla). **Mecanismo:** encontrar $a \in A$ tal que, para algún $f \in F$, $f(a) = c$ y c está asociado a una entidad.

Tipo	Descripción
Offline	Se conocen f y c ; se prueban distintos a hasta que $f(a) = c$. Ataca la función <i>complementaria</i> . Ej: obtener <code>/etc/shadow</code> (Linux) y usar <code>crack / john the ripper</code> ; obtener el archivo <code>SAM</code> (Windows) con <code>ophcrack</code> .
Online	Se usa la función L , probando distintos a hasta pasar la autenticación. Ataca la función de <i>autenticación</i> (el login). Ej: probar la función de login con una cuenta conocida (root, administrator, guest), o atacar el endpoint con herramientas tipo Postman/inspector.

Mejores ataques (no fuerza bruta pura). Asumiendo que las claves no son aleatorias:

- Diccionarios de palabras; diccionarios de claves *descubiertas* (leaks/breaches históricos).
- Transformaciones simples: sufijos, prefijos, reemplazos ($l \rightarrow 1$, $o \rightarrow 0$, vocales por números), palabras espejadas, dos palabras combinadas.
- Información de contexto: nombre, usuario, DNI, fecha de nacimiento (la peor de todas: el nombre de usuario).

1.4.1. Prevenciones

General (offline): esconder información para que a , c o f no sean conocidas simultáneamente. Como f suele conocerse (algoritmos estándar, reversing), es más fácil proteger c (ej. `/etc/shadow` solo accesible por root) que a (en manos del usuario).

Prevenir el uso de L (online):

- Tiempos crecientes ante fallas (**exponential back-off**): tras fallar se aumenta el tiempo de reintento exponencialmente.
- Deshabilitar principales que están siendo forzados (*cuidado*: un malicioso puede afectar la **disponibilidad** de un usuario haciéndolo bloquear).

- Jailing / Honeypot: al detectar ataque, se switchea L por una falsa; se deja al atacante en un ambiente controlado mientras se lo rastrea (“bote de miel”).
- Captchas (cada vez más vulnerados por bots e IA; baratos, ofrecidos como servicio).

Otras: revisión proactiva de claves (rechazar claves fáciles, buscar en diccionarios/leaks, detectar patrones, usar info del usuario), expiración/rotación de claves (limita el daño de una clave comprometida; evitar reuso de las últimas N ; *no* forzar cambio al momento de login; avisar con anticipación), subir la complejidad de las claves.

△ Cuidado — error típico

La **rotación de contraseñas** mal hecha es peligrosa: si se fuerza el cambio justo al ingresar (sin preaviso), el usuario apurado elige una contraseña fácil y **perjudica** la seguridad en vez de ayudarla. Siempre dar preaviso (7, 5, 1 día).

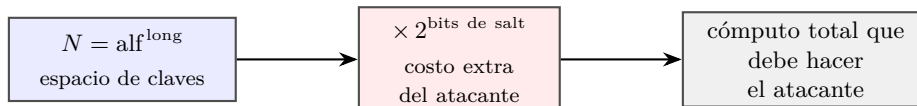
1.5. Fuerza bruta: fórmula de Anderson y metodología

Definición: Fórmula de Anderson

$$P = \frac{TG}{N}$$

donde P = probabilidad de adivinar una clave, G = número de pruebas realizables por segundo, T = tiempo dedicado a las pruebas, N = tamaño del espacio de claves. Sirve para estimar el tiempo de un ataque.

El **espacio de claves** es $N = (\text{tamaño del alfabeto})^{\text{longitud}}$.



El espacio de claves crece como alfabeto^{longitud}. El salt no agranda ese espacio, pero multiplica el cómputo del atacante offline por 2^{bits de salt} (no puede paralelizar entre cuentas).

► En el parcial

EL ejercicio que “entra siempre”. El profe lo dijo: es fácil de hacer, fácil de chequear y permite “darle vueltas de rosca”. Es la **segunda unidad más recurrente**.

Metodología paso a paso:

1. Identificar las variables: alfabeto ($\Rightarrow N = \text{alf}^L$), G (pruebas/seg), T (tiempo), P (probabilidad objetivo) y la incógnita.
2. Plantear $P \geq \frac{TG}{N}$ y despejar la incógnita.
3. **Pasar todo a la misma unidad.** Tiempo: 1 año = $365 \cdot 24 \cdot 60 \cdot 60$ s. Probabilidad objetivo típica 0,5 o 1/100.
4. Si la incógnita es la longitud L , despejar N y luego $L \geq \log_{\text{alf}}(N)$, redondeando **hacia arriba**.

Ejemplo resuelto (PDF, alfanuméricas 36 caracteres). ¿Qué longitud mínima reque-

rir para que $P < 0,5$ de hallar la clave en menos de 1 año, con $G = 100,000$ claves/seg?

$$P \geq \frac{T G}{N} \Rightarrow N \geq \frac{T G}{P} = \frac{(365 \cdot 24 \cdot 60 \cdot 60) \cdot 100,000}{0,5} \quad (1)$$

$$N = 36^L \geq 6,3 \cdot 10^{12} \quad (2)$$

$$L \geq \log_{36}(6,3 \cdot 10^{12}) \approx 8,22 \Rightarrow L \geq 9 \quad (3)$$

Otros ejemplos del PDF (despejando T):

- Numéricas de 10 dígitos ($N = 10^{10}$), $G = 10,000/\text{seg}$, $P > 0,5$: $T \leq \frac{P N}{G} = \frac{0,5 \cdot 10^{10}}{10,000} = 500,000 \text{ s} \approx 6 \text{ días}$.
- 8 letras (26 caracteres, $N = 26^8$), $G = 10,000/\text{seg}$, $P > 0,5$: $T \leq \frac{0,5 \cdot 26^8}{10,000} \approx 1,044,135 \text{ s} \approx 121 \text{ días}$.

► En el parcial

Variantes reales de parcial:

- Alfabeto $[a-zA-Z0-9]$ (62 símbolos), 10^4 passwords/seg: ¿5 símbolos y 5 bits de salt dan $P < 0,5$ en un año? (2015, 2025-1Q).
- Alfabeto 96 caracteres, $10^5/\text{seg}$, $P = 1/100$ en 365 días: hallar longitud mínima (Recup 2025).
- TPU con 700 claves/seg, clave de 12 bits: tiempo para $P = 1/100$ (Recup 2023). Con clave de 12 bits, $N = 2^{12}$.

Cuando la clave se da **en bits**, $N = 2^{\text{bits}}$. Conviene trabajar con potencias de 2 o logaritmos.

△ Cuidado — error típico

Errores que penalizan en el ejercicio de fuerza bruta:

- No pasar todo a la **misma unidad** (mezclar bits vs. símbolos vs. tiempo). 1 año = 365 días = $365 \cdot 24 \cdot 60 \cdot 60 \text{ s}$.
- Olvidar redondear L **hacia arriba** (entero).
- Invertir mal la desigualdad al despejar.

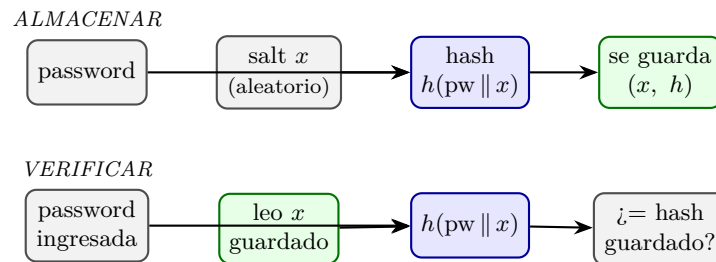
1.6. Salt (salting)

Escenario: un atacante consigue $c_1, c_2, \dots, c_n$ (una base robada de información complementaria). Sin salt, cada prueba $a_i \rightarrow f(a_i)$ puede compararse *en paralelo* contra todos los c_j .

Método: introducir una perturbación distinta por cuenta:

$$f(a) = x \parallel f'(a, x)$$

donde x es el salt (elegido aleatoriamente y embebido en el hash). Cada c se calcula con una perturbación diferente, por lo que el atacante **no puede reutilizar** $f(a)$ para buscar varias cuentas en paralelo, y debe obtener el salt de cada usuario.



La contraseña nunca se guarda: se almacena (salt, hash). Al verificar se recalcula el hash con el mismo salt y se compara. El salt hace que dos passwords iguales den hashes distintos.

! Importante — remarcado en clase

Qué hace y qué NO hace el salt (aclaración del profe):

- **NO** agranda el espacio de claves del usuario.
- **SÍ** penaliza al atacante *offline*: equivale a sumar bits al cómputo del atacante, aumentando su costo por $2^{\text{(bits de salt)}}$, porque tiene todos los hashes y debe probar más (no puede paralelizar ni detectar patrones de claves repetidas).
- Evita que dos contraseñas iguales den la misma información complementaria.

△ Cuidado — error típico

Error típico: decir que el salt **aumenta el espacio de búsqueda/de claves del usuario**. **Falso**. El salt no cambia el espacio de claves del usuario; penaliza al atacante (multiplica su costo por $2^{\text{bits de salt}}$). Por eso en el ejercicio “5 símbolos + 5 bits de salt” el salt suma al *cómputo del atacante*, no al alfabeto/longitud de la clave.

1.6.1. Funciones lentas: PBKDF2 (PKCS 5 v2.0)

Idea: encarecer el cómputo para que ejecutar el algoritmo sea costoso (suelen tardar ≥ 100 ms en vez de 10 ms), de modo que probar muchas passwords sea lento para el atacante, pero irrelevante para el sistema (lo corre una sola vez al validar).

Sea *pass* una contraseña, *S* un salt y PRF una función pseudoaleatoria (criptosistema simétrico o MAC). Se aplica iterativamente:

$$u_1 = \text{PRF}(pass, S) \quad (4)$$

$$u_2 = \text{PRF}(pass, u_1) \quad (5)$$

$$\vdots \quad (6)$$

$$u_j = \text{PRF}(pass, u_{j-1}) \quad (7)$$

$$\text{Resultado} = u_1 \oplus u_2 \oplus \dots \oplus u_j \quad (8)$$

Algoritmo completo: $K = \text{PBKDF2}(prf, pass, salt, c, len)$, con $K = T_1 \| T_2 \| \dots \| T_n$ ($n = len/|prf|$) y cada bloque $T_i = u_1 \oplus u_2 \oplus \dots \oplus u_c$, donde

$$u_1 = \text{PRF}(Pass, Salt \| \text{INT32_BE}(i)), \quad u_k = \text{PRF}(Pass, u_{k-1}).$$

! Importante — remarcado en clase

El parámetro *c* (cantidad de **rondas**/iteraciones) es ajustable: a más rondas, más caro para el atacante. Como el sistema solo lo calcula una vez (al guardar o al verificar), no importa cuánto tarde; al atacante sí le complica las cosas. El XOR final busca uniformizar

los bits (sin patrones detectables). Ya está implementado en estándar: en Java, JCE provee PBKDF2WithHmacSHA1 vía `SecretKeyFactory` (no hay que implementar nada).

► En el parcial

¿Es válido guardar passwords con SHA-256 en una BD? (Recup 2025). Discutir:

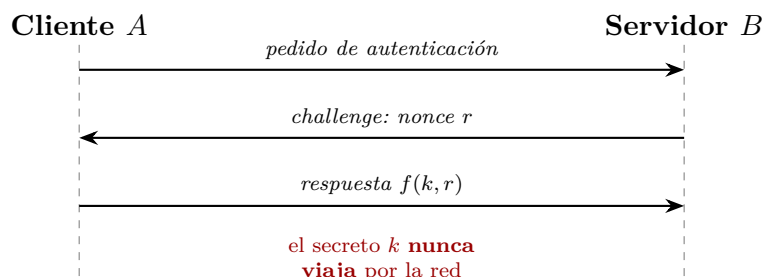
- Hashear es mejor que texto plano (guardás una prueba, no el secreto).
- Pero SHA-256 “a secas” es **rápido** $\Rightarrow$ vulnerable a fuerza bruta/diccionario/rainbow tables.
- Falta **salt**: sin él, dos claves iguales dan el mismo hash y el ataque se paraleliza.
- Lo correcto es una **función lenta** con salt y rondas configurables (PBKDF2, bcrypt, scrypt). Conclusión: SHA-256 solo *no* es adecuado.

1.7. Challenge-Response y EKE

Problema: autenticar a un usuario remoto requeriría enviar la clave (necesita canal seguro; descubrir una comunicación antigua puede comprometer la clave).

Solución (challenge-response): no enviar la clave.

1. $A \rightarrow B$: pedido de autenticación.
2. $B \rightarrow A$: un challenge $\{r\}$.
3. $A \rightarrow B$: $f(k, r)$ (aplica una función preacordada sobre su clave k y el challenge r).



Challenge-response: el servidor manda un desafío único r (nonce); el cliente responde $f(k, r)$ probando que conoce el secreto k sin enviarlo. El nonce evita reutilizar respuestas capturadas.

Así el usuario valida que conoce el secreto sin enviarlo. (Las *passkeys* actuales siguen esta idea: el servidor manda un challenge, el cliente lo firma con una clave protegida por biometría/PIN, y el servidor verifica; hay un esquema de clave pública/privada de fondo y no se envía ningún dato secreto.)

EKE (Encrypted Key Exchange). Objetivo: impedir que un atacante adivine claves capturadas en la red. Mecanismo: hacer el diálogo en un **canal encriptado** (con clave k_s), de modo que el atacante **no tiene forma de saber si una clave es correcta**.

$$A \rightarrow B : \{\text{pedido}\}_{k_s} \quad B \rightarrow A : \{r\}_{k_s} \quad A \rightarrow B : \{f(k, r)\}_{k_s}$$

1.8. OTP — Claves de uso único

△ Cuidado — error típico

No confundir OTP (One Time Password) con One Time Pad. Misma sigla, nada que ver.

Objetivo: generar claves que sirven una única vez. A y B comparten una clave K y un contador C . Concepto: $(K, C) \rightarrow \text{OTP} \rightarrow (otp, C')$. La clave K es criptográfica, generada aleatoriamente (es lo que se escanea en el QR / se carga en el authenticator). Dos variantes estandarizadas:

1.8.1. HOTP — RFC 4226 (basado en HMAC)

$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA1}(K, C))$$

Truncate extrae 32 bits de los 160 del MAC:

- $\text{offset} = \text{mac}[19] \& 0xf$ ($\leftarrow$ 4 bits del último byte)
- $\text{bin} = \text{mac}[\text{offset}..\text{offset}+3] \& 0x7fffffff$
- $\text{Result} = \text{bin} \% 10^n$ (n = cantidad de dígitos)

El **contador se incrementa en cada uso** $\Rightarrow$ requiere sincronización (**look-ahead**: el servidor calcula varios intentos hacia adelante). Se aconseja **throttling**. Genera códigos de 1 a 9 dígitos (recomendado ≥ 6).

1.8.2. TOTP — RFC 6238 (basado en tiempo)

$$\text{TOTP}(K) = \text{HOTP}(K, T), \quad T = \frac{(\text{current unix time} - T_0)}{X}$$

donde T_0 = tiempo de referencia (usualmente 0) y X = segundos de vida de cada código (usualmente 30). **No requiere contador** (el “contador” lo da el tiempo). Se aconseja un **drift-window** (cantidad de ticks desfasados aceptados, hacia atrás y hacia adelante, por des-sincronización entre servidores) y **throttling**. Es el más usado hoy (Google Authenticator, etc.).

! Importante — remarcado en clase

La complejidad frente a fuerza bruta es la misma aunque el código rote: si son 6 dígitos, se puede probar por fuerza bruta igual. Por eso se necesita throttling del lado del servidor. Los **recovery codes** son códigos de un solo uso para recuperar la cuenta si se pierde el dispositivo del segundo factor.

1.9. Multifactor y multicanal

Multi factor authentication: requerir dos (o más) factores de *distinto tipo*. Racional: cada tipo de factor requiere comprometer distintos *assets*. Combinaciones comunes: conozco+tengo, conozco+soy, tengo+soy.

► En el parcial

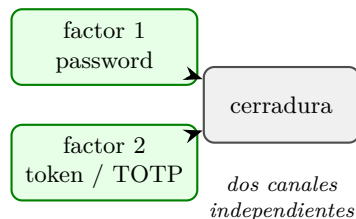
Independencia de factores (concepto clave). El multifactor es más seguro porque obliga a comprometer **varios canales independientes**. PERO si los factores están **entrelazados**, se anula la independencia y pierde sentido.

- Ejemplo del profe: el **celular** concentra “algo que tengo” + “algo que soy” (ca-

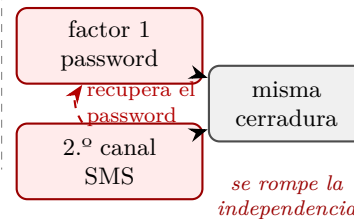
ra/huella) en un solo dispositivo. Si lo roban/clonan/comprometen, comprometen *ambos* factores a la vez.

- Ejemplo de entrelazamiento que rompe la independencia: poder **recuperar el password por el segundo canal** (SMS) $\Rightarrow$ “dos llaves para la misma cerradura”, ya no hay independencia real.

MULTIFACTOR (ok)



MULTICANAL entrelazado



Izq.: dos factores independientes, hay que comprometer dos canales distintos. Der.: si el 2.º canal (SMS) sirve para recuperar el password, ambos factores quedan entrelazados —“dos llaves para la misma cerradura”— y se pierde la independencia.

△ Cuidado — error típico

Error típico: decir que multifactor “es seguro” sin notar que **factores entrelazados rompen la independencia**. Hay que señalar la dependencia (segundo factor recuperable por el primero, ambos en el mismo dispositivo, etc.).

! Importante — remarcado en clase

Trilema usabilidad / performance / seguridad. “Seguridad siempre es un balance, algo vas a penalizar.”

— dicho en clase Pedir el segundo factor todo el tiempo es seguro pero golpea la usabilidad; por eso existe la **step-up authentication**: pedir un factor adicional solo ante anomalías (contexto extraño, cambio de geolocalización, primer login en mucho tiempo, etc.).

1.10. Autenticación remota (SSO / Single Sign-On)

Consiste en **delegar la autenticación** a un sistema externo (relación de confianza). Conviene cuando otra empresa lo hace de forma más robusta y uno quiere enfocarse en su negocio.

- **Productos propietarios:** Active Directory Federation Services (ADFS), CAS, Siteminder. (El ITBA usa Active Directory: los usuarios viven en el AD como recursos.)
- **Tecnologías abiertas:** OpenID 1 y 2 $\rightarrow$ obsoletos. **OpenID Connect** $\rightarrow$ basado en OAuth2.

! Importante — remarcado en clase

OpenID Connect vs OAuth2. OAuth2 es **autorización**; OpenID Connect le **añade la capa de autenticación** encima de OAuth2. “Login con Google/Facebook/Apple” delega la autenticación: la parte de saber *quién sos* (nombre, foto) es OpenID Connect; pedir acceso a tus recursos (drive, archivos) es OAuth2. Así no hay que crear un usuario nuevo en cada sistema externo (federated login).

1.11. Lectura recomendada

Capítulos 12–13 de *Computer Security: Art and Science*, Matt Bishop.